CHAPTER-1

INTRODUCTION

1.1 OVERVIEW

The Amniotic Fluid Index (AFI) is a critical parameter in obstetrics used to assess the amount of amniotic fluid surrounding a fetus. Amniotic fluid serves numerous essential functions, including cushioning the fetus, regulating temperature, and allowing for proper growth and movement. An abnormal AFI, either too low (oligohydramnios) or too high (polyhydramnios), can indicate potential complications such as fetal growth restriction, preterm birth, or congenital anomalies. Therefore, accurate and timely assessment of AFI is vital for optimal prenatal care.

Traditionally, AFI is measured manually via ultrasonography, where the uterus is divided into four quadrants and the vertical depth of the largest fluid pocket in each is measured. However, this method is prone to human error, operator dependency, and variability across practitioners. Moreover, static measurements do not reflect the dynamic nature of amniotic fluid levels, which can change rapidly based on maternal hydration, placental function, and fetal activity.

In this context, artificial intelligence (AI) presents a transformative opportunity. Recent advances in machine learning, deep learning, and real-time image processing can be leveraged to automate and improve the accuracy, consistency, and predictive capabilities of AFI measurement. The integration of AI in fetal health monitoring can thus enhance clinical decision-making and patient outcomes.

There is an increasing demand in obstetric practice for tools that provide **real-time, non-invasive, and accurate assessments** to reduce maternal and fetal morbidity and mortality. The current reliance on episodic ultrasound scanning fails to provide continuous insight into fetal well-being. Moreover, in under-resourced or rural healthcare settings, access to skilled sonographers and radiologists may be limited.

The proposed project aims to address these challenges by developing an **AI-driven, real-time analysis system for the Amniotic Fluid Index (AFI)**. This system will analyze ultrasound images or video streams to provide continuous, accurate, and automated measurement of AFI.

1.2 OBJECTIVES

1. To Automate the Detection of Amniotic Fluid in Ultrasound Images

- Leverage deep learning techniques, particularly U-Net architecture, for semantic segmentation of ultrasound images to identify regions containing amniotic fluid.
- Reduce reliance on manual interpretation by sonographers, thus minimizing human error and inter-observer variability.

2. To Accurately Calculate the Amniotic Fluid Index (AFI)

- Develop an algorithm to automatically measure the vertical depth of fluid pockets in each of the four quadrants of the uterus.
- Sum these measurements to compute the total AFI value in centimetre's (cm), mimicking the process used in clinical practice.

3. To Classify the AFI Results into Clinical Categories

- Based on the calculated AFI, classify the results into medical risk levels:
  - **Oligohydramnios** (AFI < 5 cm)
  - **Normal AFI** (5–19 cm)
  - **Polyhydramnios** (AFI > 20 cm)
- Provide immediate diagnostic feedback to the user with simple, interpretable messages.

4. To Build a User-Friendly Web Application for Real-Time Prediction

- Design an intuitive frontend interface that allows users (doctors or technicians) to:
  - Upload ultrasound images
  - View predictions and health status in real time
- Ensure that the system is accessible and usable even by non-technical medical staff.

5. To Offer Real-Time Decision Support for Clinicians

- Integrate the system into a clinical workflow to provide instantaneous decision support, especially during prenatal check-ups.

1.3 PROBLEM STATEMENT

Amniotic Fluid Index (AFI) is a critical parameter used in prenatal care to assess the health and development of the fetus. Accurate measurement of AFI is essential for diagnosing conditions like oligohydramnios (low amniotic fluid) and polyhydramnios (excess amniotic fluid), which can lead to serious complications if not addressed in time. Currently, AFI measurement relies on manual interpretation of ultrasound images by trained clinicians. This manual process is:

- Time-consuming
- Subjective, leading to inter-observer variability
- Prone to human error
- Often inaccessible in rural or low-resource healthcare settings due to a shortage of skilled personnel

Given these challenges, there is a pressing need for an automated, fast, and reliable method for evaluating AFI to assist healthcare professionals in making accurate and timely clinical decisions.

Therefore, this project aims to develop an AI-driven system capable of:

- Automatically detecting and segmenting amniotic fluid regions in ultrasound images using deep learning
- Computing the AFI accurately
- Delivering real-time results via an easy-to-use web interface

Such a system would standardize AFI evaluation, reduce the burden on medical staff, and improve the quality of maternal and fetal care—particularly in under-resourced or remote healthcare environments.

CHAPTER - 2

LITERATURE REVIEW

2.1 "Automated ultrasound assessment of amniotic fluid index using deep learning"

Author: Hyun Cho et al. (2021)

The key step is the segmentation of the AF pocket using our proposed deep learning network, AF-net. AF-net is a variation of U-net combined with three complementary concepts - atrous convolution, multi-scale side-input layer, and side-output layer. The experimental results demonstrate that the proposed method provides a measurement of the amniotic fluid index (AFI) that is as robust and precise as the results from clinicians. The proposed method achieved a Dice similarity of 0.877±0.086 for AF segmentation and achieved a mean absolute error of 2.666±2.986 and mean relative error of 0.018±0.023 for AFI value.

2.2 "Deep Learning-Based Computer-Aided Classification of Amniotic Fluid Levels"

Author: Almalki et al. (2022)

The study was implemented using Python programming language (ver. 3.7.12). Several libraries were used for developing the model such as PIL (ver. 8.4.0), TensorFlow (ver. 2), Numpy (ver. 1.22.0), and Matplotlib (ver. 2.2). The experiments were performed on the Google Colab with the GPU setting. The dataset was divided using five-fold cross-validation for training and testing. The training dataset was further divided into training and validation. The model optimization was performed using the Adam optimizer. Five transfer learning models were trained such as MobileNet, InceptionResnetV2, DenseNet121, ResNet50V2, and Xception.

2.3 "Fetal Biometry and Amniotic Fluid Volume Assessment End-to-End Automation Using Deep Learning"

Author: Horgan et al. (2023)

Fetal biometry and amniotic fluid volume assessments are two essential yet repetitive tasks in fetal ultrasound screening scans, aiding in the detection of potentially life-threatening conditions. In this study, we were able to create a successful end-to-end method to automatically estimate FB and AFV from ultrasound cine-loops using the ISUOG quality criteria for standard biometry planes, with results that were similar to those of expert

operators. These two tasks are part of the six fundamental items listed by the ISUOG in the recently updated practice guideline for the routine mid-trimester scan. They allow early detection of life-threatening conditions such as FGR, oligohydramnios, and polyhydramnios that are associated with increases in the risk of fetal mortality by respectively 19, 5. The study developed and prospectively assessed deep learning models for automated fetal biometry and amniotic fluid volume measurements, demonstrating human-level performance in various fetal ultrasound tasks.

2.4 "U-Net Tuning Hyperparameter for Segmentation in Amniotic Fluid Ultrasonography Image"

Author: Ayu Pradipta et al. (2022)

This paper analyzes several uses of hyperparameters to determine the performance of the U-NET model architecture, especially for segmenting AF, and finds the best performance of U-Net in segmenting amniotic fluid with a combination of RMSprop optimizer parameters. Ultrasound Amniotic Fluid (AF) images generally have image quality similar to other 2-D ultrasound images, which have noise, blurry edges, artifacts, and low contrast. Some of the confusing factors in pocket AF segmenting comprise (a) reverberation artifact, (b) AF mimicking region, (c) floating matters, and (d) incomplete or missing boundary. Obtaining the Region of Interest (ROI) area of amniotic fluid requires a segmentation method that can identify each object in more detail.

2.5 "Amniotic Fluid Classification and Artificial Intelligence: Challenges and Opportunities"

Author: Nida Aslam et al. (2022)

This paper attempted to provide a comprehensive review of the existing studies in the area of utilizing AI for AF detection and classification to provide a starting point for researchers to identify the knowledge gaps in this area and conduct more extensive research. Furthermore, it summarizes the use of DL- and ML-based classification and segmentation methods in measuring AF volume and highlights the challenges and opportunities in this field. Additionally, it discusses the various factors and causes leading to abnormal AF levels and reviews the studies focusing on using AI techniques to investigate these factors.

2.6 "Amniotic Fluid Segmentation Based on Pixel Classification Using Local Features"

Author: Sri Hartati et al. (2021)

In this paper, we propose the pixel-wise classification method for the segmentation of amniotic fluid US images to resolve this problem. We extract information more specifically at the pixel level, which will become the training data for this method. With window/kernel sampling, each pixel in the image is observed for its gray level value against its neighbor. Each area of the object is taken several samples according to the window/kernel size. The border and the fuzzy regions in the amniotic fluid are the focus of this pixel sampling. The feature values we extract are based on the local window of each pixel. We propose a novel feature, namely Distance Angle Pixel (DAP), which calculates the distance or difference pixels at each angle. We combined this feature with the gray-level local window and Gray level local variance (GLLV) values. Furthermore, we built a classification model by evaluating several methods such as Random Forest, Naïve Bayes, Decision Tree (C.45), SVM, and K-Nearest Neighbors. As a classifier, the random forest was found to be effective in medical image analysis . Naïve Bayes is used in medical image segmentation tasks .

2.7 "AFNet Algorithm for Automatic Amniotic Fluid Segmentation"

Author: Alejo Costanzo et al. (2020)

In This study presents a novel and unique approach to automated segmentation of amniotic fluid based on fetal MRI using the AFNet model, an improved version of the state-of-the-art ResUNet++ architecture. By replacing upsampling blocks with transpose convolutional blocks and average pooling layers with atrous convolutional blocks, AFNet demonstrates enhanced performance in AF segmentation. Our experiments revealed that utilizing Adagrad with a high learning rate was a more effective optimization strategy for this network. Furthermore, we observed improved performance by training usingwhole images with a smaller batch size instead of cropping. In summary, our AFNet model significantly advances automated amniotic fluid segmentation based on fetal MRI. Its novel architectural modifications, competitive performance, and consistent results position AFNet as a valuable tool for accurate and efficient AF segmentation.

2.8 “Deep Learning-Based Detection and Classification of Amniotic Fluid Echogenicity Type”

Author: Putu Desian et al. (2025)

All The examination of amniotic fluid focuses on two aspects: the volume of the fluid and its echogenicity. The echogenicity is caused by the presence of meconium-stain (feses) and vernix caseosa (a protective layer on the fetal skin) . Meconium-stain and vernix caseosa are two significant components found in amniotic fluid, each serving distinct roles in fetal development . Meconium, a sterile mixture produced in the fetal intestines, is composed of water, mucopolysaccharides, and solid materials such as vernix caseosa and lanugo . Conversely, vernix caseosa, a protective substance covering the fetal, plays an important role in the skin’s barrier function and may also serve as a site for dioxin excretion, highlighting its importance in the detoxification process . Research indicates that vernix contains branched-chain fatty acids, which are absent in meconium, suggesting that vernix may have a unique metabolic function. Meconium-stain (feces) is detected in the amniotic fluid, there is concern that the fetal may develop Meconium Aspiration Syndrome (MAS), a condition in which feces enter the lungs, potentially leading to fatal complications, fetal distress, and indicating underlying issues such as hypoxia or asphyxia, which can elevate the risk of perinatal morbidity and mortality . Our proposed model designed by applying a CNN based semantic segmentation approach to amniotic fluid images, with modifications made to the dense layer. Pretrained models incorporating various CNN architectures, are utilized to extract features from meconium-stain and vernix caseosa. Feature selection is carried out using three methods: Chi-Square, ANOVA, and Mutual Information. Xtreme Gradient Boosting algorithm for classification. Our proposed model achieve accuracy of 0.94 or 94% for classifying echogenicity type.

2.9 “Deep learning-based model for Amniotic Fluid Segmentation in 2D fetal Ultrasound images”

Author : Rihem Mahmoud, Taha Reheh et al. (2025)

Automatic computerized segmentation of fetal anatomical structures from ultrasound images and biometric measurements is still challenging, due to the inherent characteristics of fetal ultrasound images at different semesters of pregnancy. Generally, clinicians manually measure the fetal anatomical structures by fitting either an ellipse (AF-Pocket plane or a line (for the case of femoral plane), and it can take considerable time to take

these measurements. The difficulty in automated fetal anatomical structures measurement stems from various uncertain factors, including poor and non-uniform contrast across the border of the fetal structures, the high variability of the ultrasound images, and large intersect variation. It is extremely difficult to extract the edge of the fetal anatomical structures using local patterns alone. In this paper, we propose an automated method for AF-Pocket semantic segmentation using deep learning methods. The proposed method contains four steps: Pre-processing, binary classification, semantic segmentation and AF-Pocket measurement. The results of our proposed architecture were evaluated using a prepared dataset and achieves the best results compared with a variety of state-of-the-art deep learning-based approaches.

2.10 "Artificial intelligence in obstetric ultrasound: A scoping review"

Author: Rabecca Horgan et al. (20225)

The objective is to summarize the current use of artificial intelligence (AI) in obstetric ultrasound. PubMed, Cochrane Library, and ClinicalTrials.gov databases were searched using the following keywords "neural networks", OR "artificial intelligence", OR "machine learning", OR "deep learning", AND "obstetrics", OR "obstetrical", OR "fetus", OR "foetus", OR "fetal", OR "foetal", OR "pregnancy", or "pregnant", AND "ultrasound" from inception through May 2022. The search was limited to the English language. Studies were eligible for inclusion if they described the use of AI in obstetric ultrasound. Obstetric ultrasound was defined as the process of obtaining ultrasound images of a fetus, amniotic fluid, or placenta. AI was defined as the use of neural networks, machine learning, or deep learning methods. The authors' search identified a total of 127 papers that fulfilled our inclusion criteria.

LITERATURE SURVEY

SL NO:	TITLE OF THE PAPER	YEAR	AUTHORS	CONTENT EXTRACTED FROM THE PAPER
1	Automated Ultrasound Assessment of Amniotic Fluid Index Using Deep Learning	2020	Hyun Chao	This study introduces a deep-learning-based method for automatic AFI evaluation from ultrasound images. The proposed method achieved a Dice similarity coefficient of 0.877_+0.086 for amniotic fluid segmentation.
2	Deep Learning-Based Computer-Aided Classification of Amniotic Fluid Levels	2020	Almalki	Utilizing transfer learning models like Xception and MobileNet, this paper classifies AF levels as normal or abnormal using ultrasound images. The MobileNet model achieved an accuracy of 94%, highlighting the potential of AI in aiding sonographers.
3	Fetal Biometry and Amniotic Fluid Volume Assessment End-to-End Automation Using Deep Learning	2021	Horgan	The study developed and prospectively assessed deep learning models for automated fetal biometry and amniotic fluid volume measurements, demonstrating human-level performance in various fetal ultrasound tasks.
4	U-Net Tuning Hyperparameter for Segmentation in Amniotic Fluid Ultrasonography image	2022	Ayu Pradipta	This paper analyzes the impact of hyperparameter tuning on the performance of the U-Net model for segmenting amniotic fluid in ultrasound images, aiming to optimize segmentation accuracy.
5	Amniotic Fluid Classification and Artificial Intelligence Challenges and Opportunities	2022	Nida Aslam	This review discusses the challenges and advancements in AI-based techniques for amniotic fluid classification, emphasizing the need for standardized datasets and evaluation metrics.

6	Amniotic Fluid Segmentation Based on Pixel Classification Using Local Features	2023	Sri Hartati	The study proposes a pixel classification model based on window sampling for amniotic fluid segmentation in B-mode ultrasonography images, enhancing segmentation precision.
7	AFNet Algorithm for Automatic Amniotic Fluid Segmentation	2023	Alejo Costanzo	This paper introduces AFNet, a novel convolutional neural network model for segmenting amniotic fluid .
8	Deep Learning Based Model for AF Segmentation in 2Dimage	2024	Putu Desian	The study applies a CNN-based semantic segmentation approach to detect and classify amniotic fluid echogenicity types.
9	Deep learning-based model for Amniotic Fluid Segmentation in 2D fetal Ultrasound images	2025	Rihem Mahmoud, Taha Reheh	This paper presents a deep learning model for segmenting amniotic fluid in 2D fetal ultrasound images, contributing to automated AFI measurement.
10	Artificial intelligence in obstetric ultrasound: A scoping review	2025	Rabecca Horgan	The Review explores current applications of AI in obstetric ultrasound, including first-trimester scans, placental assessment, and fetal biometry.

Table 2.1

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

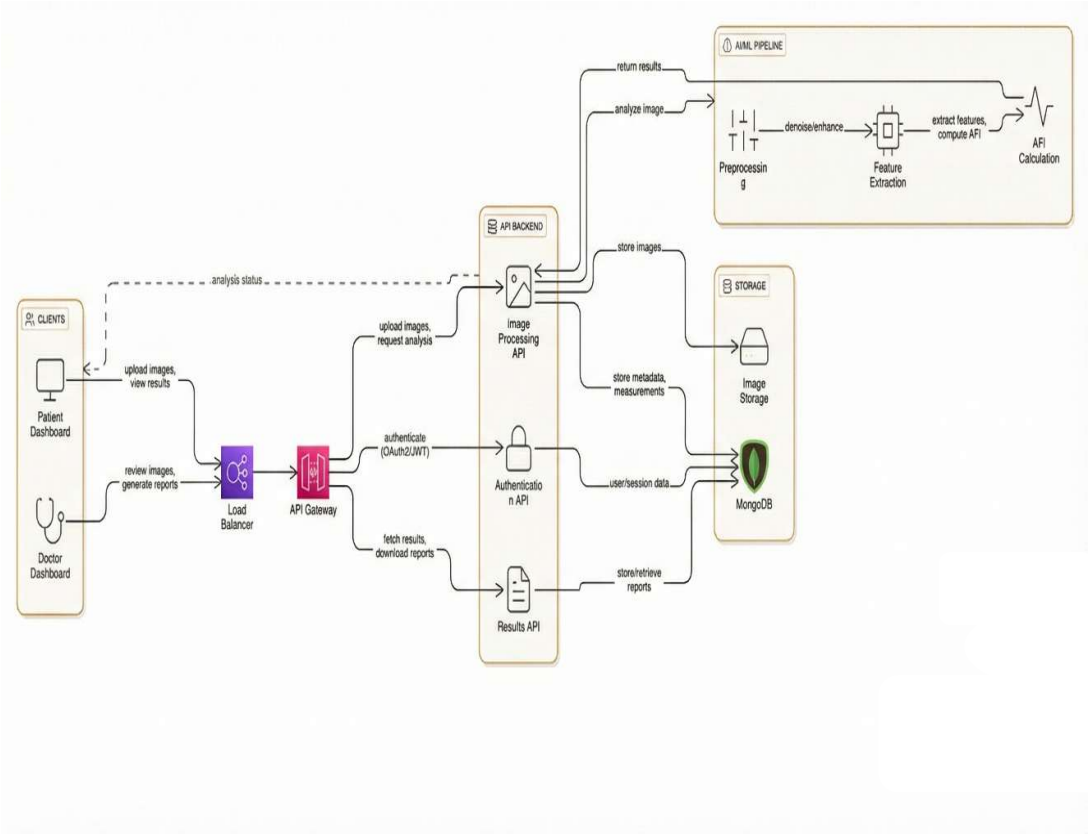


Fig 3.1 System Architecture

1. CLIENTS (Frontend Layer):

This layer represents the end-users interacting with the system through different interfaces.

- **Patient Dashboard:** Allows patients to "upload images" and later "view results." It also receives asynchronous notifications regarding "analysis status."
- **Doctor Dashboard:** Allows medical professionals to "review images" uploaded by patients and "generate reports" based on the analysis.

2. Entry Point & Routing

Traffic from the clients enters the backend through a managed gateway layer.

- **Load Balancer:** The single point of contact for client traffic. It distributes incoming network traffic across backend servers to ensure high availability and reliability.
- **API Gateway:** Sits behind the load balancer. It acts as the "front door" for all the microservices, handling routing, request composition, and protocol translation.

3. API BACKEND (Microservices Layer)

The application logic is broken down into distinct, loosely coupled microservices.

- **Authentication API:**
  - Handles security. It receives credentials to "authenticate" users using standards like **OAuth2/JWT** (JSON Web Tokens).
  - It stores and manages "user/session data" in the PostgreSQL database.
- **Image Processing API:**
  - The core orchestrator for imaging workflows. It receives requests to "upload images, request analysis."
  - It manages the flow of data to storage and the AI pipeline (detailed below).
  - It provides the "analysis status" back to the client.
- **Results API:**
  - Dedicated to data retrieval. It handles requests to "fetch results, download reports."
  - It queries the database to "store/retrieve reports" for the frontend dashboards.

4. AI/ML PIPELINE (Processing Layer)

This is an asynchronous processing chain dedicated to analyzing the medical images. It is triggered by the Image Processing API sending an "analyze image" request. The pipeline consists of three sequential steps:

1. **Preprocessing:** The raw image undergoes cleaning, such as "denoise/enhance" operations, to prepare it for analysis.
2. **Feature Extraction:** The system identifies and isolates relevant structures or characteristics within the preprocessed image.
3. **AFI Calculation:** Based on the extracted features, the system performs final computations. "AFI" likely refers to a specific medical metric (e.g., Automated Fundus Image analysis for eye health).

   Once complete, the pipeline will "return results" to the Image Processing API.

5. STORAGE (Data Persistence Layer)

The system uses a **polyglot persistence** architecture, meaning it uses different types of storage best suited for different data types.

- **Image Storage (Object Storage):** Represented by the bucket icon (likely a service like AWS S3 or Azure Blob Storage). This is used to "store images" (the large, unstructured binary files) sent by the Image Processing API.

- **PostgreSQL (Relational Database):** A structured SQL database used to store transactional and metadata. It holds:
    - "store metadata, measurements" (results from the AI pipeline).
    - "user/session data" (from the Authentication API).
    - Reports data for the Results API.

Summary Workflow Example: An Image Upload

1. A Patient uploads an image via their dashboard.
2. The request goes through the Load Balancer and API Gateway to the Image Processing API.
3. The API saves the raw file to Image Storage.
4. Simultaneously, the API sends the image data to the AI/ML Pipeline.
5. The pipeline preprocesses the image, extracts features, calculates the AFI, and returns the results to the API.
6. The Image Processing API saves these results ("metadata, measurements") into PostgreSQL.
7. Later, a Doctor logs in (via Auth API), requests to review the data. Their dashboard calls the Results API, which fetches the structured data from PostgreSQL and displays the report.

3.2 FLOWCHARTS OF EACH INTERFACE

3.2.1 Patients Dashboard Flowchart

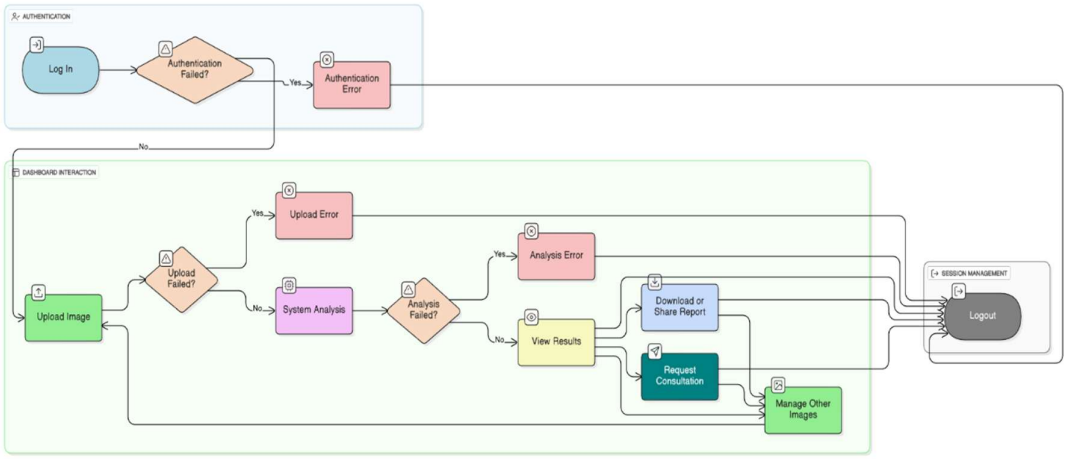


Fig 3.2.1 Patients Dashboard Flowchart

1.Authentication

- Log In: The user starts by attempting to log into the system.

- Authentication Failed? : The system checks if the login credentials are correct.
    - Yes: If authentication fails, the user is directed to an Authentication Error screen, and the process ends there.
    - No: If authentication is successful, the user proceeds to the Dashboard Interaction section**.**

2. Dashboard Interaction

This is the main functional area where the user performs tasks.

- Upload Image: The user uploads an image for analysis.
- Upload Failed?: The system checks if the upload was successful.
    - Yes: If the upload fails, the user sees an Upload Error and is then directed to Logout.
    - No: If the upload is successful, the system proceeds to System Analysis.
- System Analysis: The system processes the uploaded image.
- Analysis Failed?: The system checks if the analysis completed successfully.
    - Yes: If the analysis fails, the user sees an Analysis Error and is directed to Logout.
    - No: If the analysis is successful, the user can View Results.
- View Results: From here, the user has three options:
    - Download or Share Report: The user can save or share the analysis report. After this, they can either choose to Manage Other Images or Logout.
    - Request Consultation: The user can request a professional consultation
    - based on the results. After this, they can also choose to Manage Other Images or Logout.
    - Manage Other Images: The user can choose to handle other images. This action loops back to the Upload Image step, allowing for a new cycle of analysis.

3. Session Management

This section handles the end of the user's session:

- Logout: This is the final step where the user's session is terminated. The user is directed to Logout from several points:
    - After any error (Upload Error, Analysis Error).
    - After completing an action like Download or Share Report.

3.2.2 Doctors Dashboard Flowchart

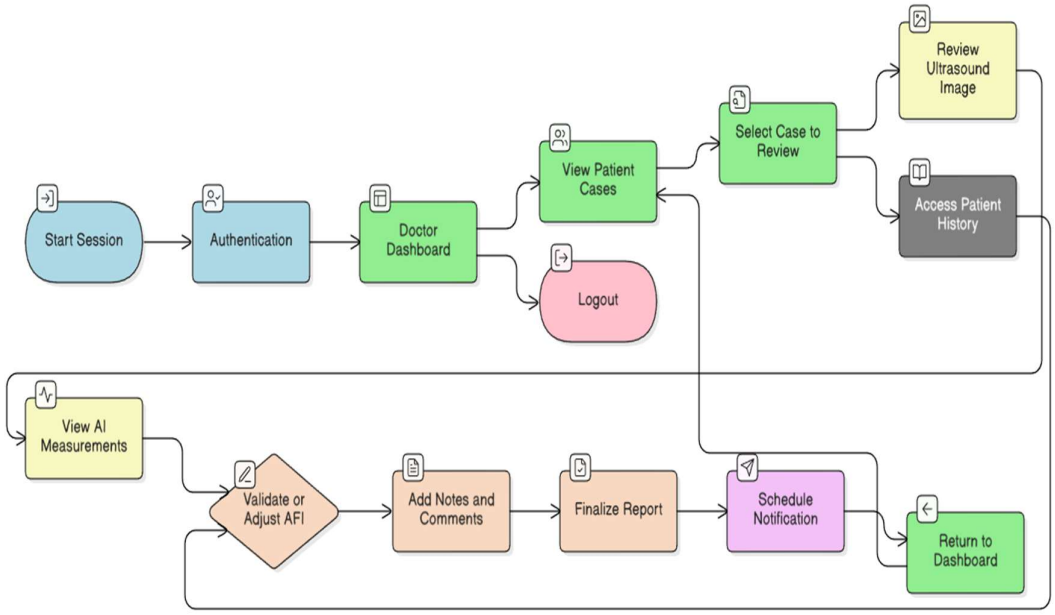


Fig 3.2.2 Doctors Dashboard Flowchart

1. Access and Dashboard (Start)

- Start Session & Authentication: The process begins with the user starting a session and undergoing "Authentication" (logging in secure).
- Doctor Dashboard: Upon successful login, the user lands on the "Doctor Dashboard," which serves as the central hub.
- From the dashboard, the user has two immediate options:
    - Logout: End the session.
    - View Patient Cases: Proceed with the main workflow.

2. Case Selection and Initial Review

- Select Case: Upon choosing "View Patient Cases," the doctor moves to "Select Case to Review" from a list.
- Parallel Review: Once a case is selected, the workflow branches into two simultaneous actions to gather context:
    - Review Ultrasound Image: Looking at the medical scans.
    - Access Patient History: Reviewing past medical records.

3. AI-Assisted Analysis and Validation

- View AI Measurements: Both review paths converge here. The doctor views measurements pre-calculated by an Artificial Intelligence system based on the ultrasound image.
- Validate or Adjust AFI (Decision Point): This is a critical step. The doctor reviews the AI's findings (specifically "AFI," likely referring to a cardiac metric like Automated Function Imaging). The circular arrow indicates an iterative process where the doctor can validate the AI's suggestions or manually adjust them until satisfied.

4. Reporting and Completion

- Add Notes and Finalize: Once the measurements are validated, the doctor moves to "Add Notes and Comments" and then "Finalize Report."
- Post-Report Actions: After finalizing, the flow branches again:
  - One path goes to "Schedule Notification" (e.g., informing the patient or referring physician) before moving to the next step.
  - The other path bypasses notification
- Return to Loop: Both paths converge at "Return to Dashboard." This action takes the doctor back to the "View Patient Cases" screen, ready to select the next patient and repeat the cycle.

3.2.3 Use Case Flow Diagram

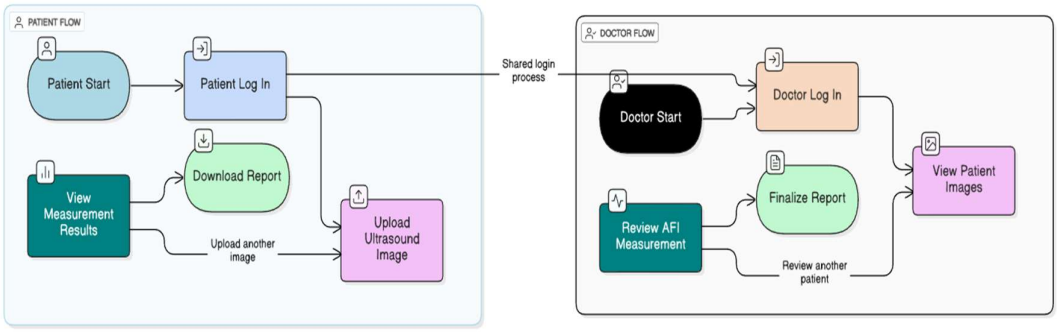


Fig 3.2.3 Use Case Flow Diagram

1. Patient Flow (Left Section)

The patient's journey focuses on data submission and retrieving results.

- Authentication: The process begins with Patient Start leading to Patient Log In.
- Data Submission: Once logged in, the primary action is to Upload Ultrasound Image.

- Analysis & Results: After the upload is processed, the patient can View Measurement Results.
- Post-Analysis Options:
    - Download Report: The patient can save a copy of the final analysis.
    - Loop (Upload another image): If the patient has more scans to analyze, the system allows them to loop back and upload a new image immediately.

2. Doctor Flow (Right Section)

The doctor's journey focuses on review, validation, and reporting.

- Authentication: The Doctor Start leads to Doctor Log In.
    - There is a connector line labeled "Shared login process" between Patient Log-in and Doctor Log In, indicating both users likely utilize the same authentication infrastructure.
- Case Review: The doctor logs in to View Patient Images (presumably those uploaded by the patients).
- Assessment: The doctor proceeds to Review AFI Measurement (Amniotic Fluid Index or similar metric).
- Actionable Outcomes:
    - Finalize Report: The doctor validates the findings and closes the case.
    - Loop (Review another patient): The workflow allows the doctor to cycle back to the image list to handle the next patient's case efficiently.

Summary of Interaction

The system acts as a bridge where:

1. Patients act as Data Contributors (uploading raw ultrasound images).
2. Doctors act as Validators/Reviewers (analyzing the pre-processed measurements and finalizing reports).

CHAPTER 4
REQUIREMENTS

4.1 SYSTEM OVERIEW

The AFI web application enables users (patients and doctors) to:

1. Upload ultrasound images for AI-based classification.
2. Authenticate securely using JWT tokens.
3. View analysis results and history.
4. Doctors can review patient data, view health trends, and manage appointments.
5. All data is stored securely in MongoDB for persistence and auditing

4.2 SOFTWARE REQUIREMENTS

4.2.1 Frontend Requirements

- **Language:** JavaScript (ES6+)
- **Framework:** React.js
- **Libraries:** Material-UI (MUI) for responsive design and icons
- **Routing:** React Router DOM
- **HTTP Client:** Axios for REST API communication
- **Visualization:** Charts, progress bars, and confidence meters for AFI insights
- **Styling:** CSS3 with gradient themes and glass morphism effects
- **Testing:** React Testing Library and Jest

4.2.2 Backend Requirements

- **Framework:** FastAPI (Python 3.10+)
- **Server:** Uvicorn ASGI Server
- **Libraries:**
    - Motor (async MongoDB driver)
    - Python-Jose for JWT Authentication

  - Passlib for password hashing
  - Python-Multipart for file uploads
  - TensorFlow/Keras for deep learning inference
  - Pillow for image processing
  - Scikit-learn and NumPy for data handling
- **Error Handling:** Structured responses and CORS configuration
- **Authentication:** JWT-based secure token system with 30-minute expiry

4.2.3 Database Requirements

- **Database:** MongoDB (Atlas / Local Instance)
- **Driver:** Motor (async)
- **Collections:**
  - Users (for login, roles)
  - Predictions (for AFI results)
  - Appointments (for scheduling)
- **Backup and SSL:** certifi added for secure TLS communication

4.2.4 Machine Learning Stack

- **Framework:** TensorFlow / Keras
- **Dataset:** Ultrasound image dataset organized into class folders
- **Classes:** Low, Normal, High AFI
- **Output:** Softmax probabilities with confidence score
- **Model Type:** CNN-based classification model (trained offline and deployed in backend)

4.3 HARDWARE REQUIREMENTS

- **Processor:** Intel i5 or higher
- **RAM:** Minimum 8GB (Recommended 16 GB)

- **GPU(Optional):** NVIDIA GTX 1650/ RTX 2050 for Tensor Flow
- **Storage:** 100 GB SSD
- **Internet:** Required for MongoDB Atlas and Cloud API's

4.4 FUNCTIONAL REQUIREMENTS

- **User Registration & Login:** Patients and doctors register and log in using JWT authentication.
- **Role-Based Access:** Doctors and patients have separate dashboards and permissions.
- **Image Upload:** Upload ultrasound images for AFI classification with validation.
- **AI-Based Prediction:** TensorFlow model classifies image into Low, Normal, or High
- **Doctor Dashboard:** Doctors can view patient AFI results, generate reports, and track statistics.
- **Appointment System:** Patients can book internal/external appointments (Practo integration).
- **Health Record Management:** Secure storage and retrieval of patient medical history.
- **Admin Panel:** Manage users, monitor predictions, and handle database health.
- **Report Generation:** Generate and download analysis reports in PDF format

4.5 NON-FUNCTIONAL REQUIREMENTS

- **Performance:** Response time < 3s on GPU; optimized for concurrency via async I/O.
- **Security:** JWT-based authentication; passwords hashed using Passlib; encrypted data storage.
- **Reliability:** Automatic retries for failed DB connections; fallback to local storage.
- **Scalability:** Stateless backend allows horizontal scaling with containers.

- **Availability:** 99% uptime target via cloud deployment.
- **Maintainability:** Modular architecture with independent React and FastAPI modules.
- **Usability:** WCAG-compliant UI; responsive for all screen size
- **Portability:** Supports Docker-based deployment for portability.
- **Data Privacy:** Follows HIPAA principles for sensitive medical information.

4.6 API REQUIREMENTS

- **POST /api/auth/register:** Register new user
- **POST /api/auth/token:** POST /api/auth/token
- **GET /api/auth/me:** Retrieve profile info
- **POST /api/prediction/predict_image:** Upload image and get AFI classification
- **GET /api/history/predictions:** Fetch all predictions
- **GET /api/history/predictions/{id}:** Fetch specific prediction
- **GET /api/prediction/download/{id}:** Download analysis report

4.7 UI/UX REQUIREMENTS

- **Design Language:** Material UI + custom gradients (#667eea, #764ba2)
- **Accessibility:** WCAG 2.1 AA compliant; supports keyboard navigation
- **Animation:** Smooth transitions using CSS keyframes
- **Visualization:** Confidence bars, AFI statistics, pregnancy tracker
- **Dark/Light Mode:** Toggle for visual comfort
- **Responsiveness:** Works on mobile, tablet, and desktop (320px–1920px width range)

4.8 MACHINE LEARNING REQUIREMENTS

- **Input:** Ultrasound image (JPEG/PNG)
- **Preprocessing:** Resizing (224×224), normalization, grayscale conversion

- **Model Type:** CNN (Convolutional Neural Network) trained on labeled AFI dataset.
- **Output:** AFI classification and confidence score.
- **Training Environment:** TensorFlow GPU backend; Python 3.10+
- **Deployment:** TensorFlow SavedModel served in backend via FastAPI ML route
- **Retraining Frequency:** Quarterly updates with new labeled data

4.9 PERFORMANCE AND TESTING REQUIREMENTS

- **Unit Testing:** Pytest (backend), Jest (frontend)
- **Integration Testing:** Validate image upload, model prediction, and database storage pipeline
- **Load Testing:** Ensure concurrent predictions do not exceed latency threshold
- **Security Testing:** Verify JWT expiration and data protection mechanisms
- **UI Testing:** Verify responsiveness across devices

4.10 MAINTAINANCE AND BACKUP

- **Database Backup:** MongoDB Atlas automatic snapshots
- **Image Backup:** Cloud object storage (S3/MinIO)
- **Code Maintenance:** GitHub version control with feature branches
- **Logging:** Structured logs with timestamps and user identifiers

4.11 ACCEPTANCE CRITERIA

The system is considered complete when:

- A user can successfully register and log in.
- An ultrasound image can be uploaded and classified accurately.
- The classification result and history are stored in MongoDB.
- The UI is responsive and performs within acceptable limits.
- Admin and doctor dashboards function correctly.

CHAPTER 5

METHODOLOGY

5.1 METHODOLOGY APPROACH

The project adopts a Hybrid Agile + Prototyping methodology, ideal for AI + full-stack applications. The development workflow involved:

5.1.1 Requirement Analysis

- Studied AFI medical standards.
- Identified core functionalities (image upload, AI inference, history, dashboards).
- Collected system, UI, and database requirements.

5.1.2 Architectural Planning

- Defined a 3-tier architecture:
  Frontend → Backend → ML Model → Database.
- Created flowcharts, DFDs, and sequence diagrams.
- Identified REST API endpoints.

5.1.3 Model Development (AI Pipeline)

- Created a CNN-based TensorFlow model for AFI classification.
- Preprocessed ultrasound images and tested model performance.

5.1.4 Backend Development

- Implemented REST APIs using FastAPI.
- Added JWT authentication and MongoDB integration.
- Implemented prediction and history endpoints.

5.1.5 Frontend Development

- Designed patient & doctor dashboards.
- Implemented file upload, result display, and history UI.
- Integrated Axios for backend communication.

5.1.6 System Integration

- Connected the React frontend to FastAPI backend.

- Connected backend to MongoDB and TensorFlow model.

5.1.7 Testing

- Performed unit, integration, and validation testing.

5.1.8 Deployment

- Prepared Docker-ready architecture for cloud deployment.

5.2 PROPOSED SYSTEM METHODOLOGY WORKFLOW

5.2.1 User Interaction Layer (Frontend - React)

- User opens the web application.
- Authentication using login/registration page.
- Patient uploads ultrasound image.
- Doctors manage appointments and view patient reports.
- Axios sends secure (JWT-attached) requests to backend.
- React displays the model prediction results and history.

5.2.2 Application Layer (Backend - FastAPI)

FastAPI serves as the core engine that handles:

a) Authentication

- JWT authentication
- Password hashing
- Role-based access (patient, doctor)

b) Image Handling

- Receives the ultrasound file
- Validates file format and size
- Preprocesses image before inference

c) Prediction Logic

- Loads TensorFlow model at startup
- Runs prediction on the uploaded image
- Generates output: *Low / Normal / High* + confidence score

d) Database Operations

Uses Motor, the async MongoDB driver, for:

- Saving prediction history
- Registering users
- Managing appointments & clinical records

5.2.3 Machine Learning Layer (TensorFlow Model)

The AI-driven AFI classification model is implemented using TensorFlow/Keras.

Model Development Steps:

1. **Dataset preparation**
    - Ultrasound images categorized into Low, Normal, High.
2. **Image preprocessing**
    - Resize
    - Normalize
    - Optional: grayscale conversion
3. **CNN Architecture**
    - Convolution + ReLU
    - Max pooling
    - Dropout (for regularization)
    - Dense layers
    - SoftMax output (3 classes)
4. **Training**
    - Run on GPU/CPU depending on availability
    - Save the model as TensorFlow Saved Model format
5. **Deployment**
    - Model loaded once during application startup
    - Optimized for real-time inference

5.2.4 Data Management Layer (MongoDB)

MongoDB stores all persistent information:

Collections Used

- users → User details & roles
- predictions → AFI results, timestamps, image paths
- appointments → Doctor booking system
- records → Optional patient medical data

Why MongoDB?

- Flexible schema
- Scalable
- Fast reads/writes
- Ideal for JSON-based apps

5.3 IMPLEMENTATION DETIALS

5.3.1 Backend Implementation

a) File Upload Handler

- Accept image file using FastAPI's Upload File
- Validate extension (jpg, jpeg, png)

b) Preprocessing Module

- Convert image to array
- Resize
- Normalize

c) Model Prediction Module

- The TensorFlow model is loaded once:

  model = tf.keras.models.load_model("ml/model/image_model")

- The image is passed through:

  pred = model.predict(image)

5.3.2 Authentication Implementation

Steps:

1. User registers → password hashed
2. User logs in → JWT token generated

3. Token used in all protected endpoints

4. Backend validates roles and permissions

5.3.3 Frontend Implementation

a) React Component Structure

- PatientDashboard.jsx
- DoctorDashboard.jsx
- UploadComponent.jsx
- HistoryComponent.jsx
- Login/Register pages

b) Axios Services

- authService.js
- predictionService.js

c) Result Display

After prediction:

- Confidence meter
- Color-coded category
- Report download option

5.3.4 UI Enhancements (from your UI Transformation Notes)

- Gradient backgrounds
- Glassmorphic cards
- Responsive layouts
- Animated transitions
- Input validation with icons
- Patient & doctor dashboards with charts

5.3.5 Database Implementation

Using Motor (Async Driver)

- Fast, scalable
- Non-blocking operations
- Ideal for real-time applications

Prediction Record Structure

```
Json
{
  "user_id": "...",
  "afi_class": "Normal",
  "confidence": 0.87,
  "timestamp": "...",
  "image_path": "uploads/img123.jpg"
}
```

5.4 INTEGRATION METHODOLOGY

5.4.1 Frontend → Backend Integration

- Axios sends POST request with JWT
- Backend processes request and returns JSON result

5.4.2 Backend → Model Integration

- Image converted to ML-ready format
- TensorFlow model predicts
- Model output parsed and stored

5.4.3 Backend → MongoDB Integration

- Backend stores prediction, user, appointment data
- Async operations ensure low latency

5.5 TESTING METHODOLOGY

a) Unit Testing

- Model loader
- Preprocessing functions
- JWT validation
- DB handlers

b) Integration Testing

- Upload → Predict → Save → Fetch
- Authentication flow testing

c) UI Testing

- Form validation
- Navigation testing

d) Performance Testing

- Prediction latency under multiple loads
- DB read/write speeds.

5.6 DEPLOYMENT METHODOLOGY

Setup Requirements

- Python virtual environment
- Node.js for frontend
- MongoDB connection string (.env)
- certifi package to fix TLS errors

Deployment Options

- Cloud VM (AWS / Render / GCP)
- Docker container orchestration
- Netlify / Vercel for frontend hosting

CHAPTER 6

CODE AND RESULTS

6.1 train_simple_model.py CODE

```
import os
import json
from pathlib import Path
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.metrics import classification_report, confusion_matrix

def prepare_data_generators(train_dir, img_size=(224, 224), batch_size=32):
    """
    Create data generators for training and validation using ImageDataGenerator.
    Uses the same augmentation settings as the notebook.
    """
    train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=20,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        validation_split=0.2
    )

    train_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        subset='training'
    )

    val_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical',
        subset='validation'
    )

    return train_generator, val_generator

def build_model(input_shape=(224, 224, 3), num_classes=3):
    """
    Build the same CNN architecture as used in the notebook.
    """
    model = Sequential([
```

```
        Conv2D(32, (3,3), activation='relu', input_shape=input_shape),
        MaxPooling2D(2,2),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D(2,2),

        Conv2D(128, (3,3), activation='relu'),
        MaxPooling2D(2,2),

        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.3),
        Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

def train_model(train_dir, model_dir=None, img_size=(224, 224), batch_size=32,
epochs=25):
    """
    Train the CNN model using data from train_dir.
    Saves the model and class mapping to model_dir.
    """
    if model_dir is None:
        model_dir = os.path.join(os.path.dirname(__file__), 'model', 'image_model')

    # Prepare data generators
    train_generator, val_generator = prepare_data_generators(
        train_dir, img_size=img_size, batch_size=batch_size
    )

    # Build and compile model
    model = build_model(input_shape=img_size + (3,),
num_classes=len(train_generator.class_indices))

    # Ensure model directory exists
    os.makedirs(model_dir, exist_ok=True)

    # Callbacks for model saving and early stopping
    # Use a checkpoint file pattern for ModelCheckpoint
    checkpoint_path = os.path.join(model_dir, 'checkpoints', 'cp-{epoch:02d}-
{val_loss:.2f}.ckpt')
    os.makedirs(os.path.dirname(checkpoint_path), exist_ok=True)

    callbacks = [
```

```
        tf.keras.callbacks.EarlyStopping(
            monitor='val_loss',
            patience=3,
            restore_best_weights=True
        ),
        tf.keras.callbacks.ModelCheckpoint(
            checkpoint_path,
            monitor='val_loss',
            save_best_only=True,
            save_weights_only=False,
            save_format='tf'
        )
    ]

    # Train the model
    history = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=epochs,
        callbacks=callbacks
    )

    # Explicitly save the final model as SavedModel format
    # This ensures the model is saved in the format expected by prediction.py
    print(f"Saving final model to: {model_dir}")
    model.save(model_dir, save_format='tf')
    print(f"✅ Model saved successfully as SavedModel to: {model_dir}")

    # Save class mapping
    class_indices = {v: k for k, v in train_generator.class_indices.items()}
    class_names = [class_indices[i] for i in range(len(class_indices))]

    labels_path = os.path.join(model_dir, 'labels.json')
    with open(labels_path, 'w') as f:
        json.dump(class_names, f)
    print(f"✅ Class labels saved to: {labels_path}")
    print(f"Class names: {class_names}")

    return model, class_names, history

def test_on_images(model_dir, test_dir, img_size=(224, 224), batch_size=32):
    """Load a saved model and evaluate it on an image test directory structured by
class."""
    model = tf.keras.models.load_model(model_dir)

    test_ds = tf.keras.utils.image_dataset_from_directory(
        test_dir,
        labels='inferred',
        label_mode='int',
        seed=123,
        image_size=img_size,
```

```
        batch_size=batch_size
    )

    # Get true labels and predictions
    y_true = []
    y_pred = []
    class_names = None
    for images, labels in test_ds:
        preds = model.predict(images)
        preds_labels = preds.argmax(axis=1)
        y_true.extend(labels.numpy().tolist())
        y_pred.extend(preds_labels.tolist())
        if class_names is None and hasattr(test_ds, 'class_names'):
            class_names = test_ds.class_names

    print("Classification report:")
    print(classification_report(y_true, y_pred, target_names=class_names))
    print("Confusion matrix:")
    print(confusion_matrix(y_true, y_pred))

if __name__ == '__main__':
    # Example usage assumptions:
    # Place images under backend/ml/data/images/<class_name>/*.png (or jpg)
    base = os.path.join(os.path.dirname(__file__), 'data', 'images')
    model_output = os.path.join(os.path.dirname(__file__), 'model', 'image_model')

    # If a 'test' subfolder exists, we'll run training then test; otherwise only train.
    train_model(base, model_dir=model_output, epochs=8)
    test_dir = os.path.join(base, 'test')
    if os.path.exists(test_dir):
        test_on_images(model_output, test_dir)
```

6.2 SCREENSHOTS

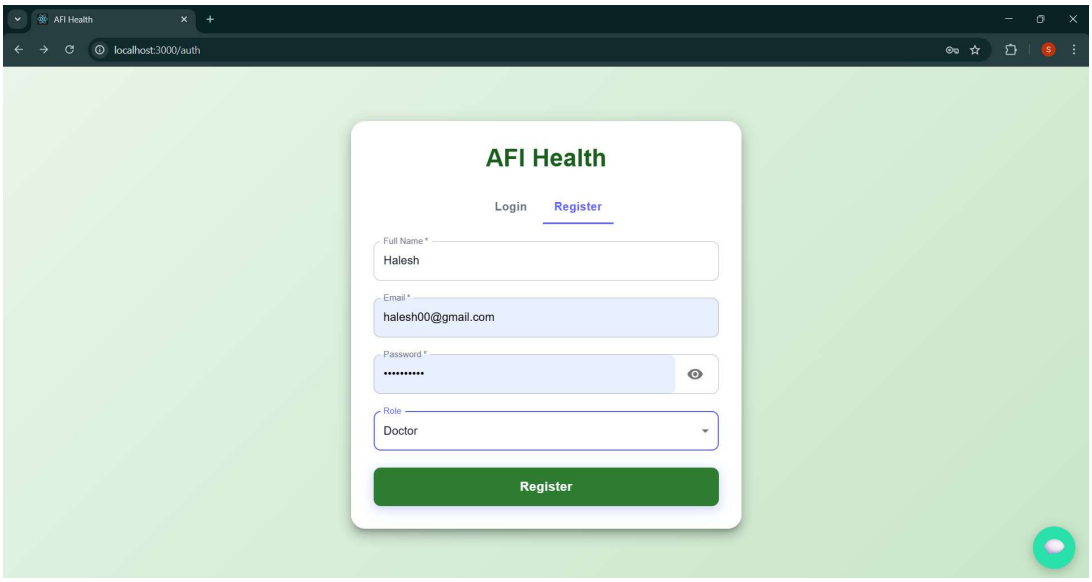


Fig 6.2.1 Register Page

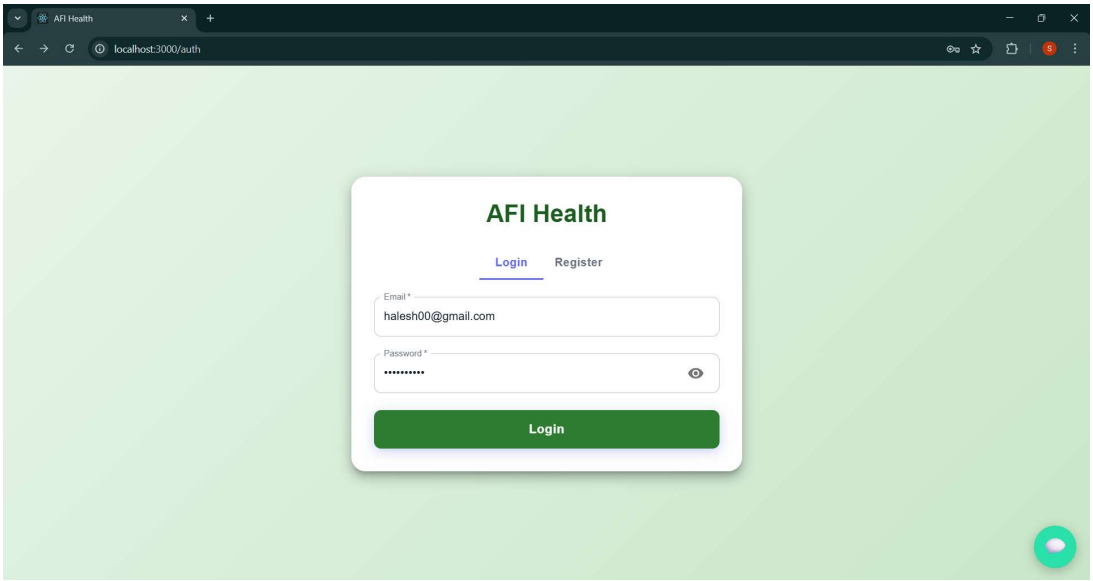


Fig 6.2.2 Login Page

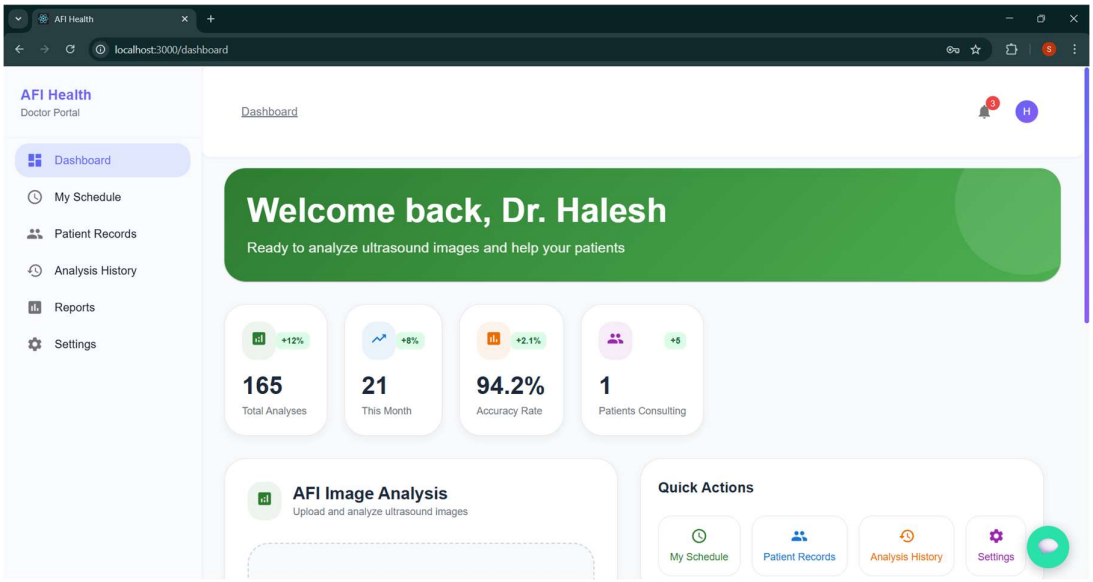


Fig 6.2.3 Doctor Dashboard

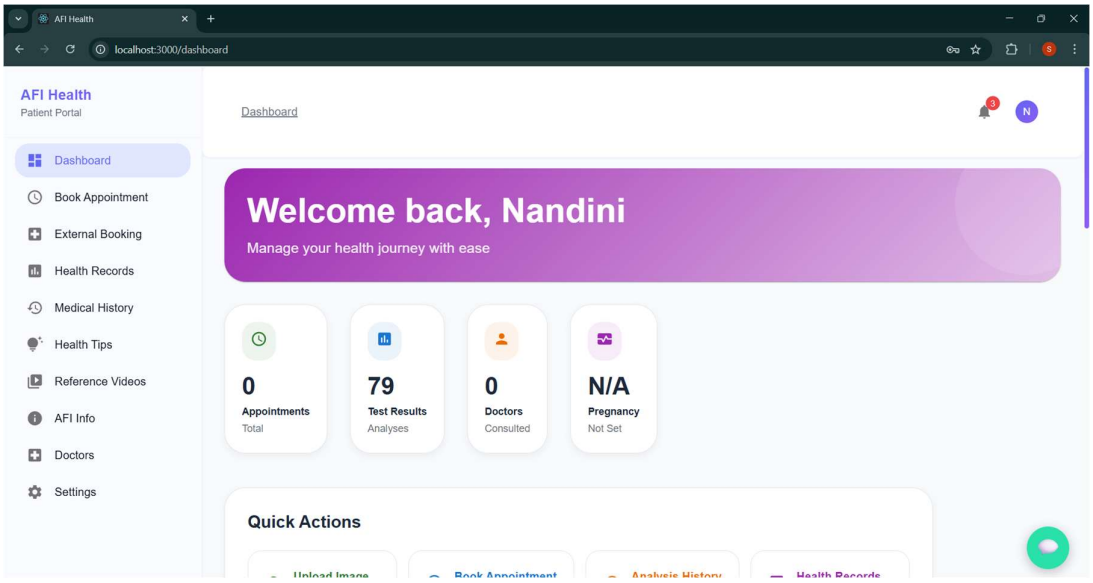


Fig 6.2.4 Patient Dashboard

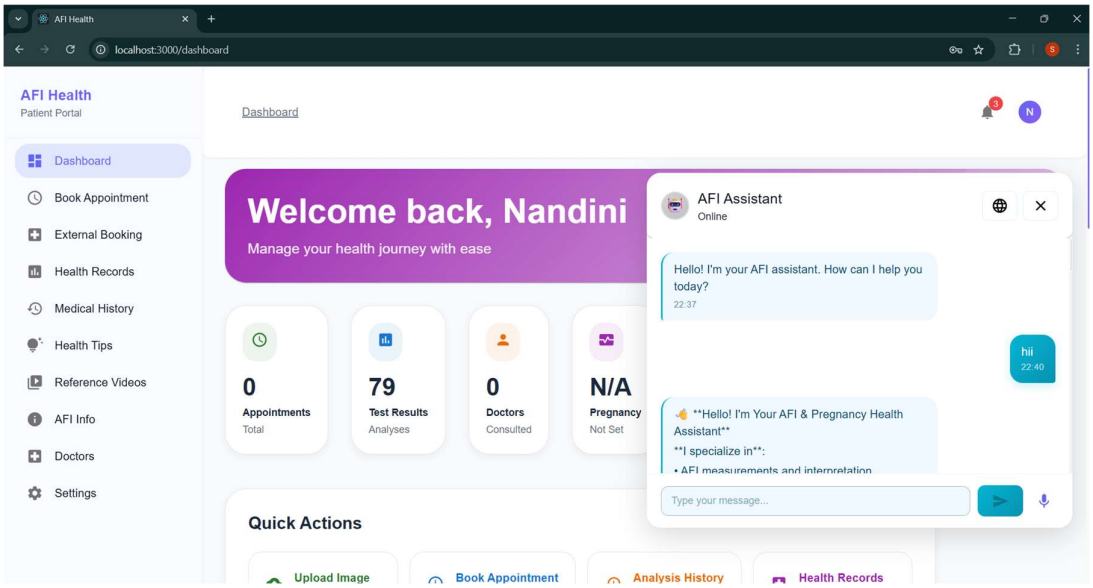


Fig 6.2.5 Patient using Chatbot

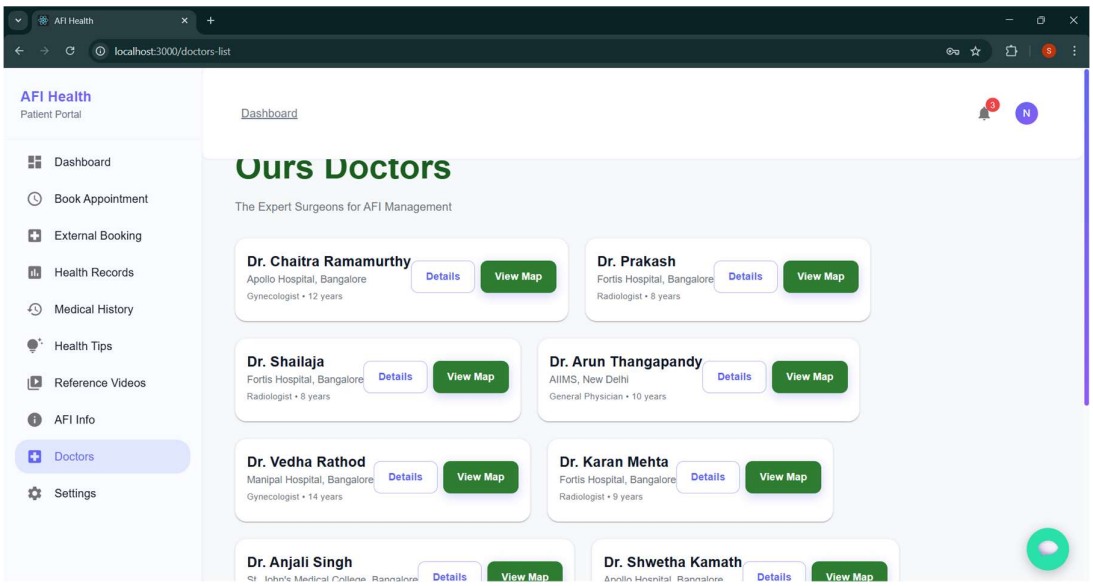


Fig 6.2.6 Doctors Details

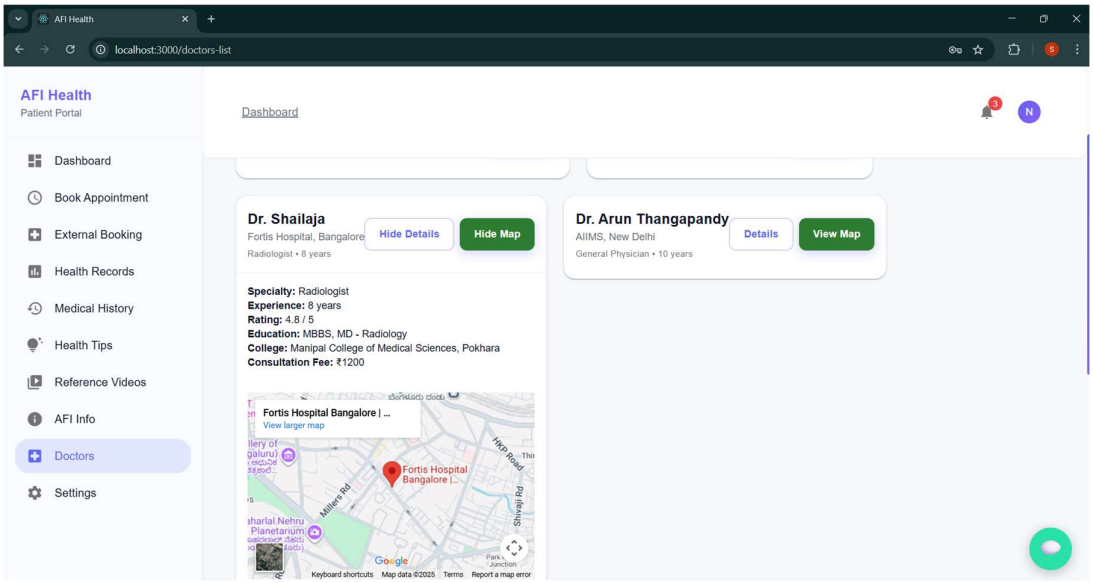


Fig 6.2.7 Patient view Doctors details

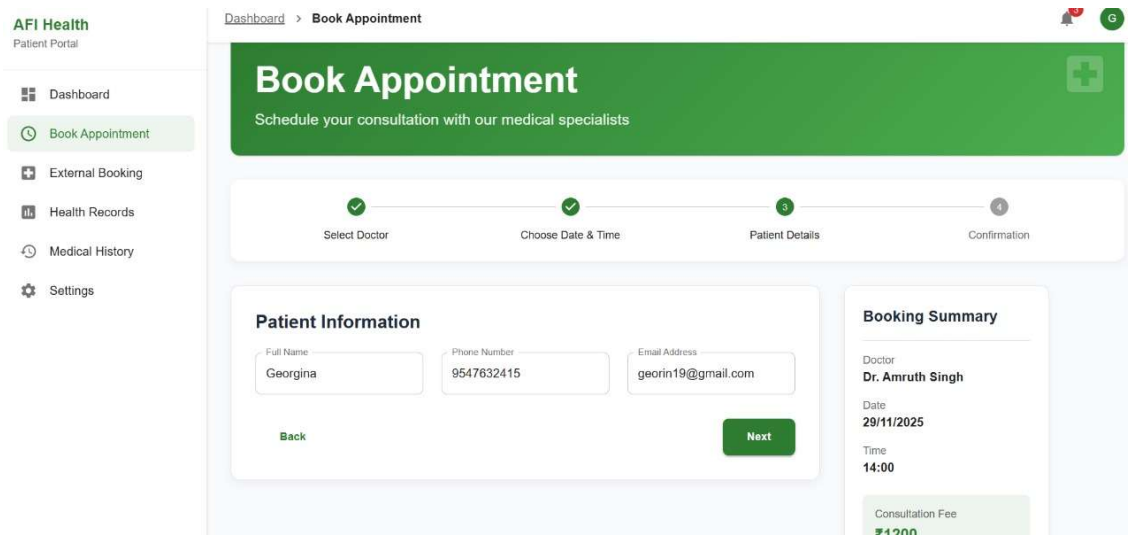


Fig 6.2.8 Appointment Booking

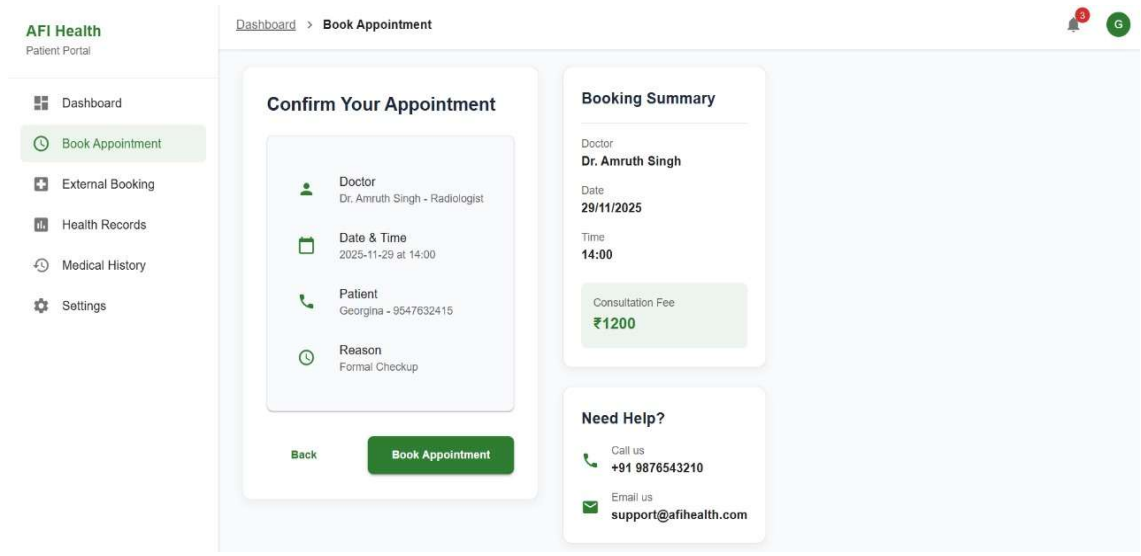


Fig 6.2.9 Booking Summary

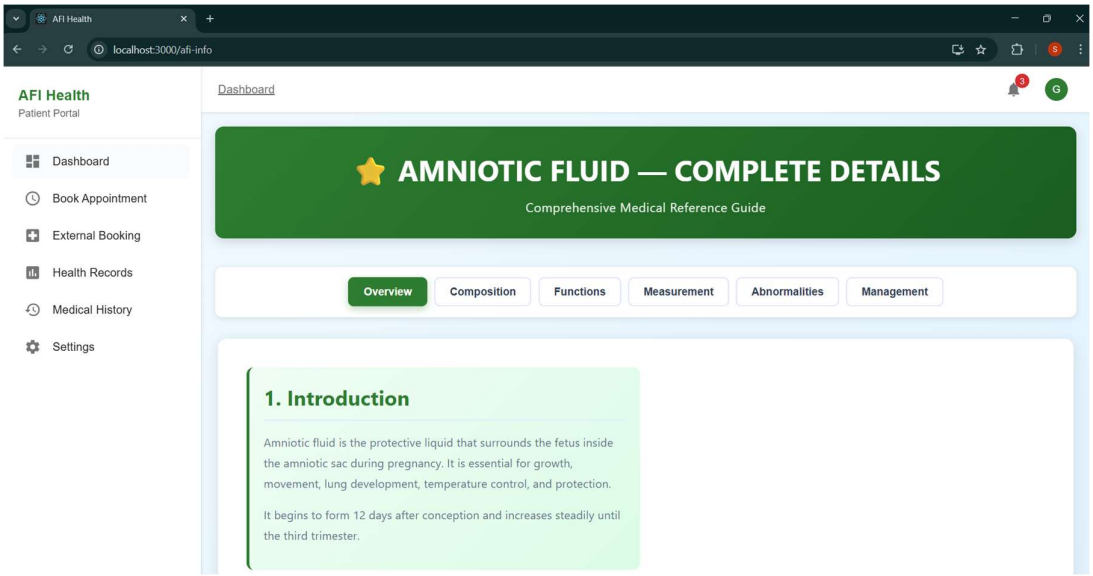


Fig 6.2.10 AFI Information

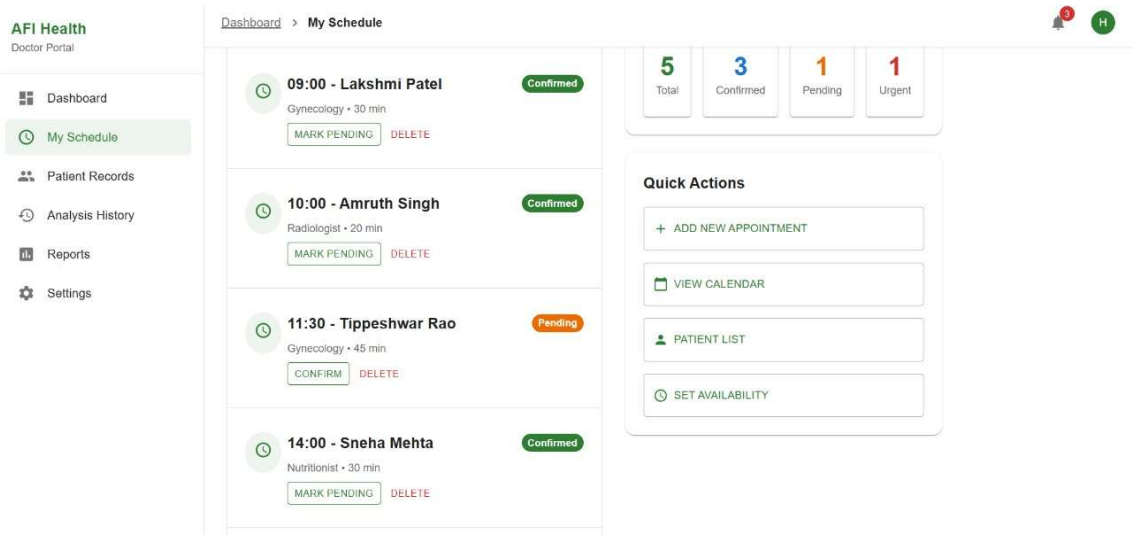


Fig 6.2.11 Doctor's Schedule

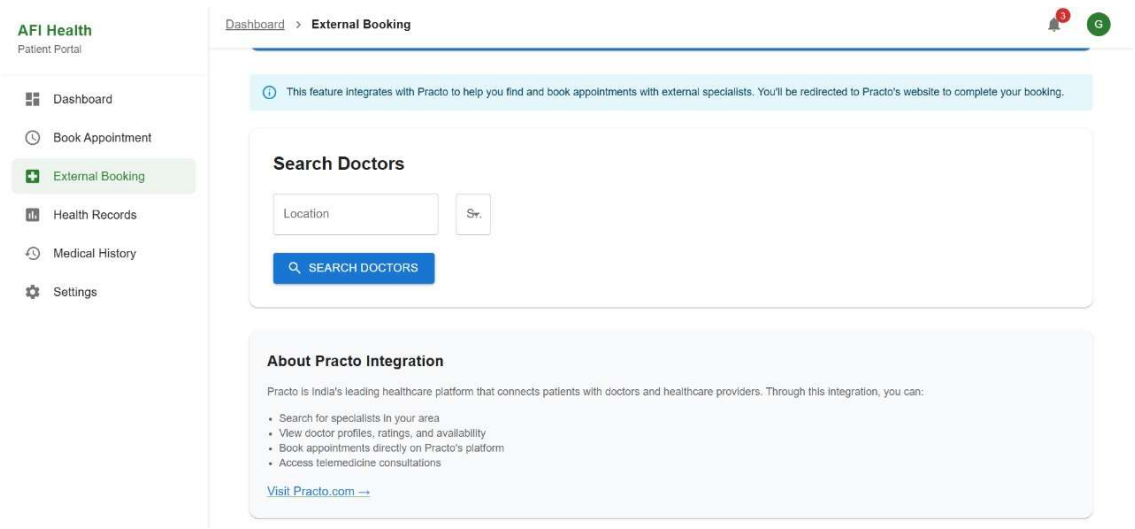


Fig 6.2.12 External Booking

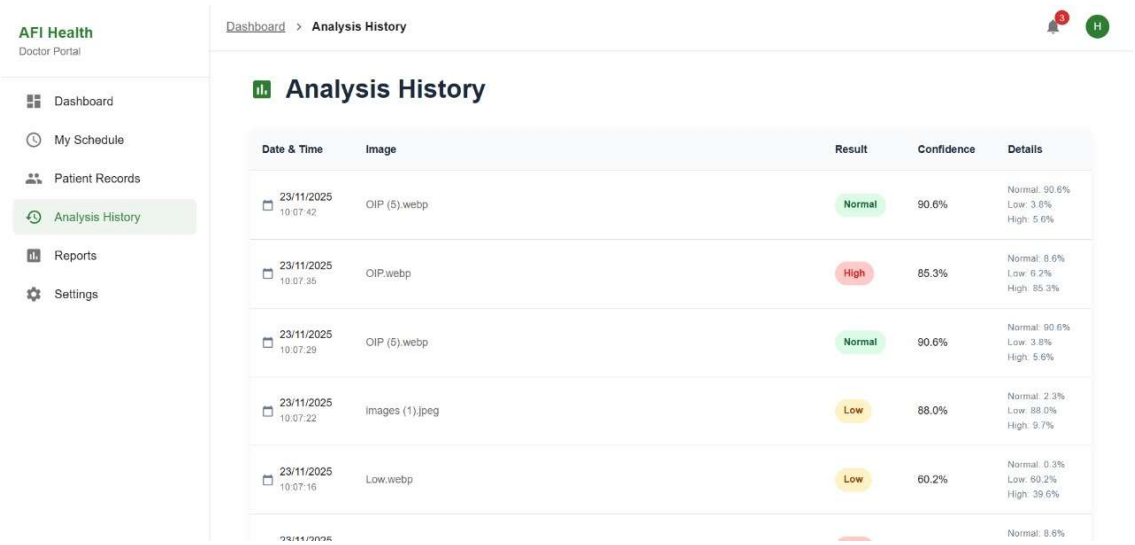


Fig 6.2.13 Analysis History

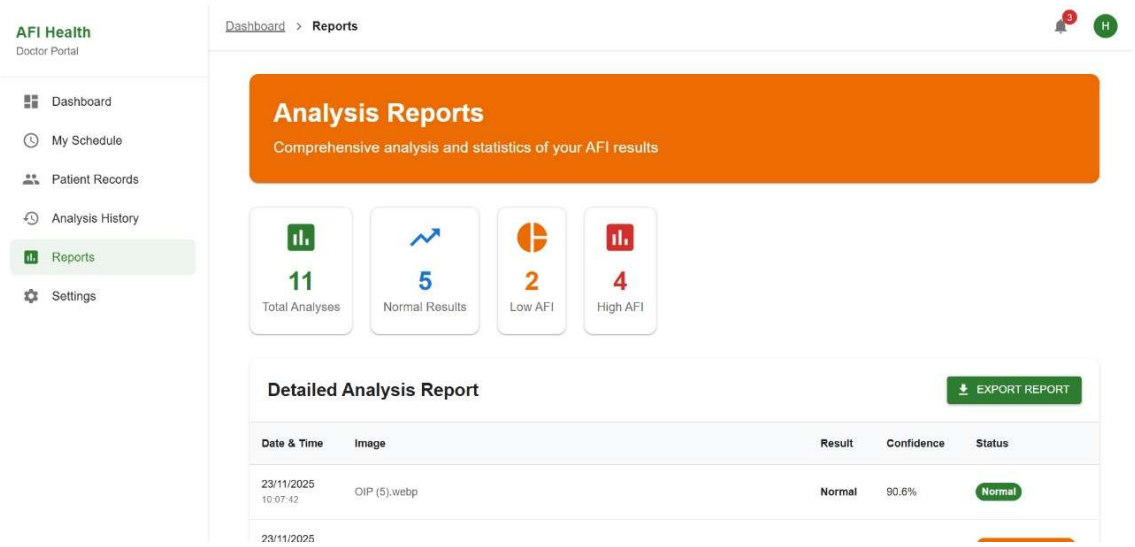


Fig 6.2.14 Patients Reports

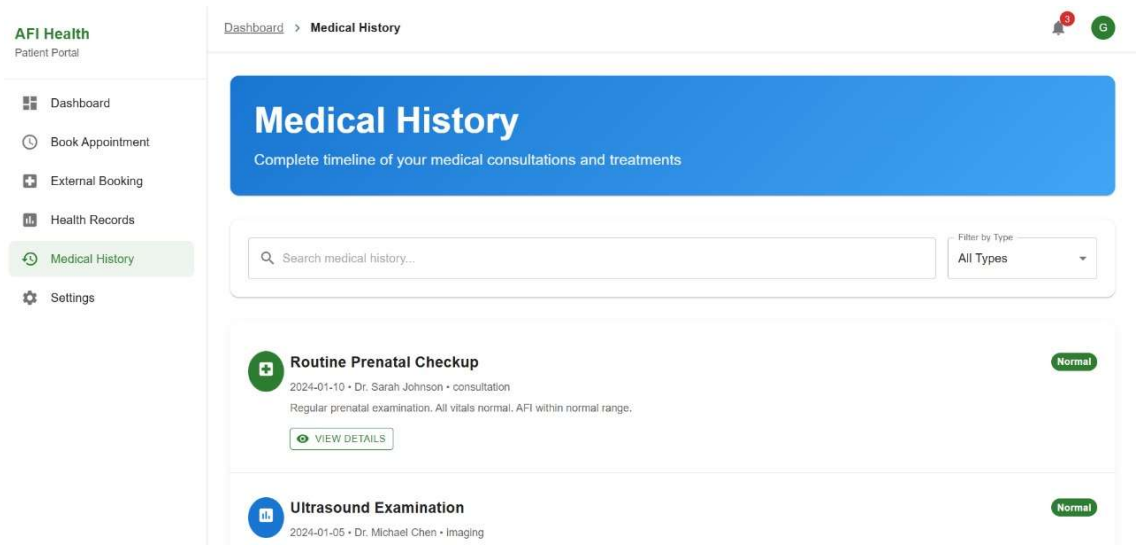


Fig 6.2.15 Medical History

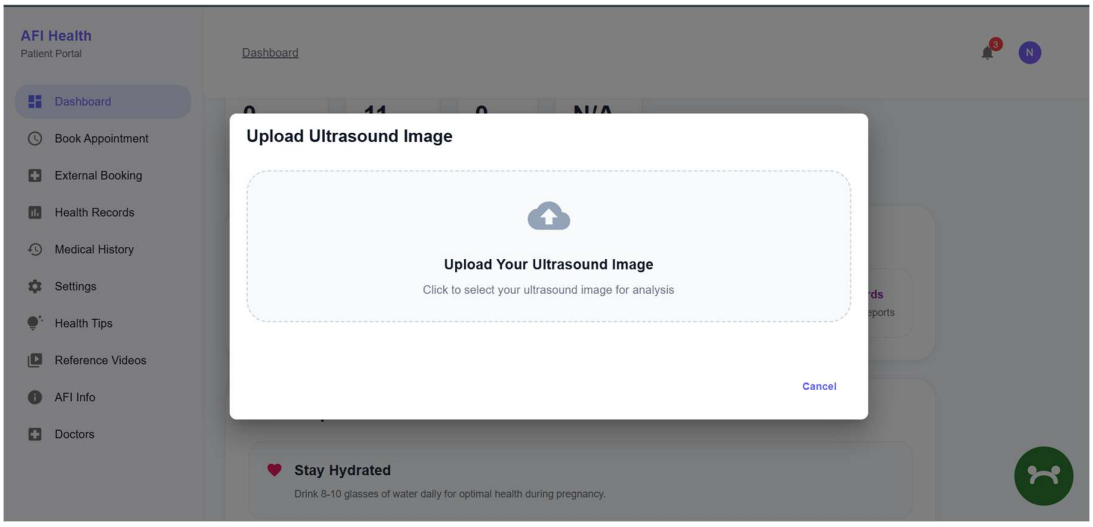


Fig 6.2.16 Upload Image

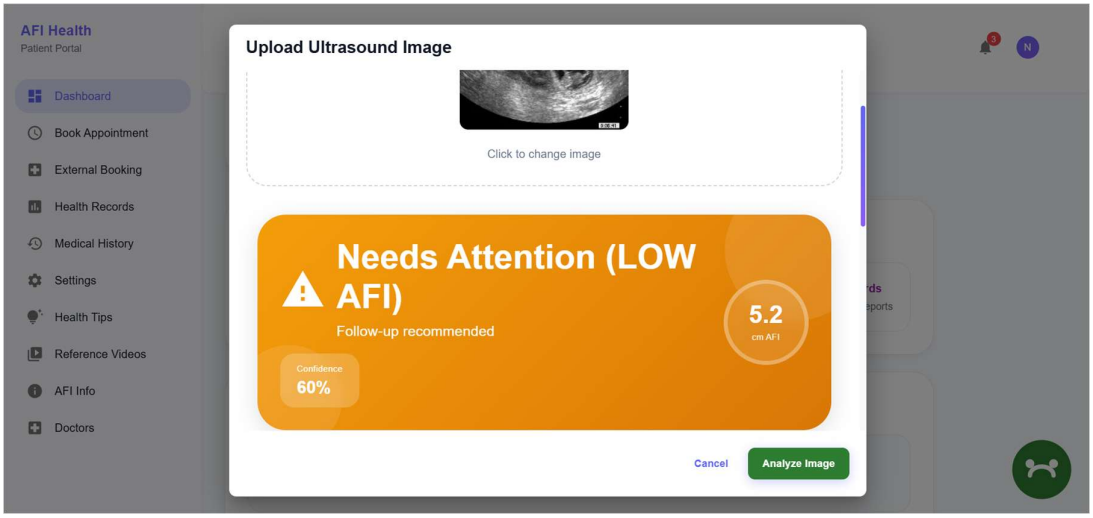


Fig 6.2.17 Analysis Result (Low)

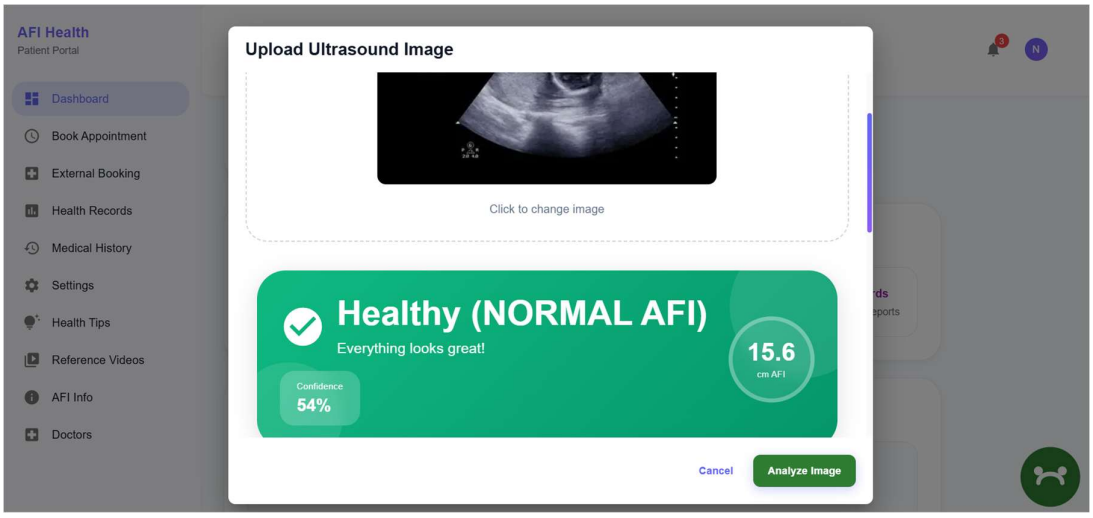


Fig 6.2.18 Analysis Results (Normal)

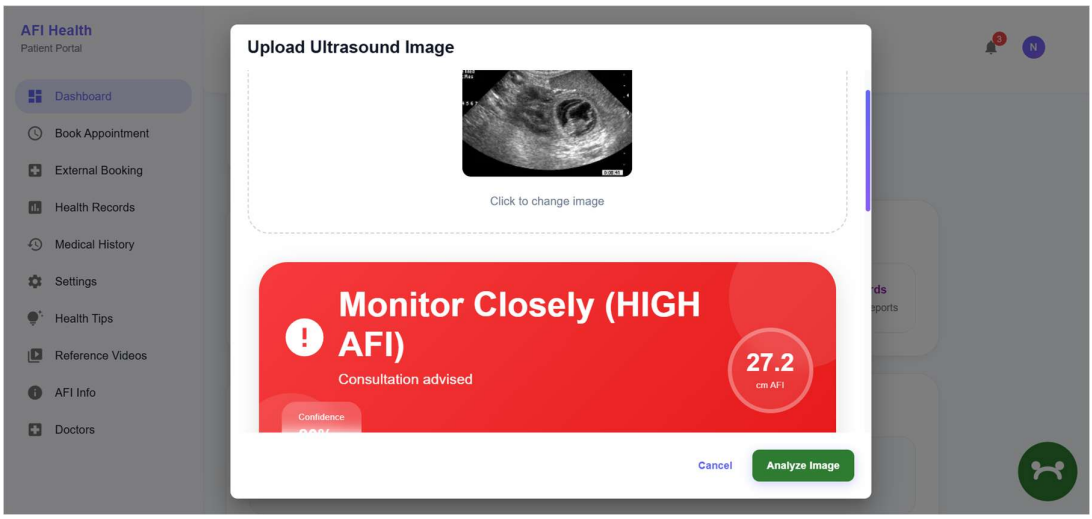


Fig 6.2.19 Analysis Results (High)

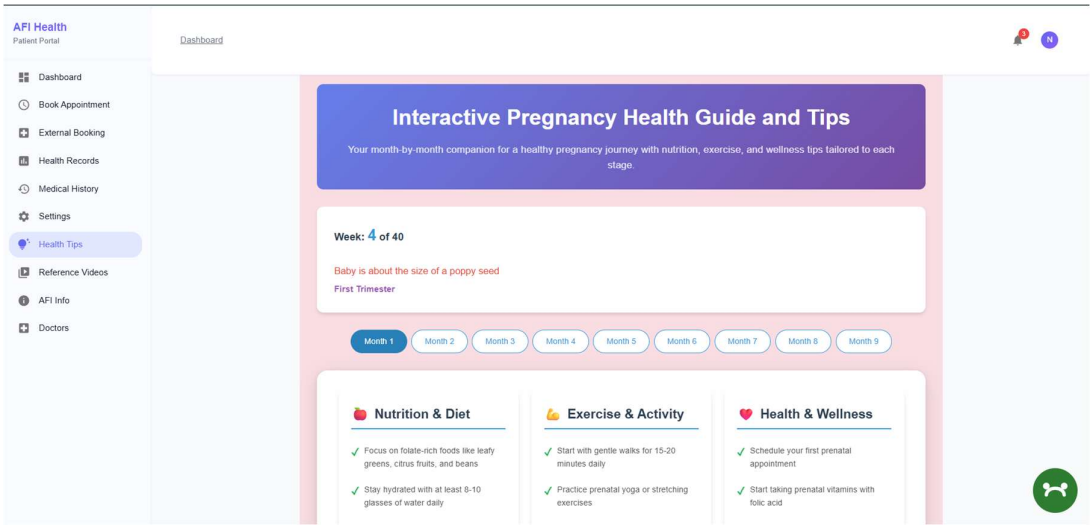


Fig 6.2.20 Health Tips

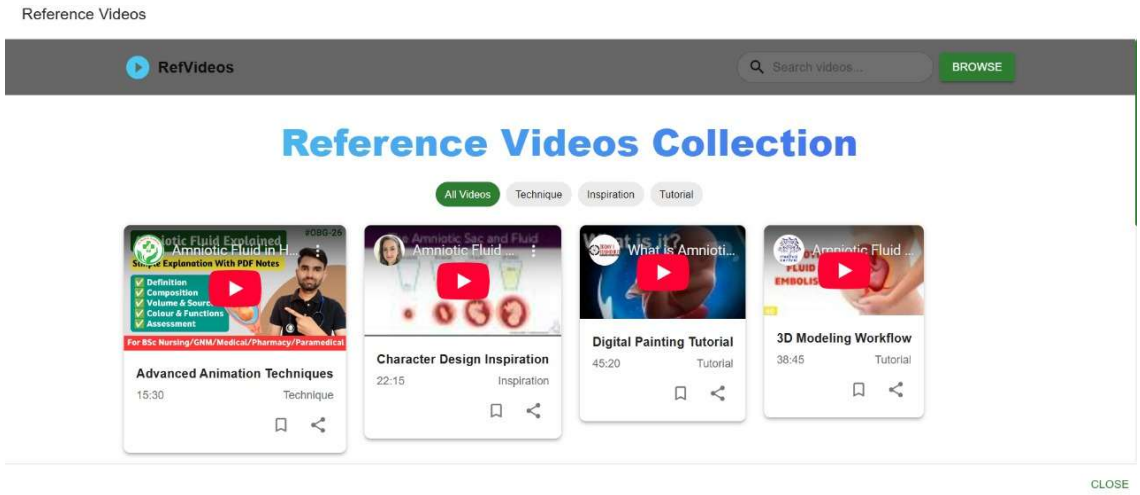


Fig 6.2.21 Reference Videos

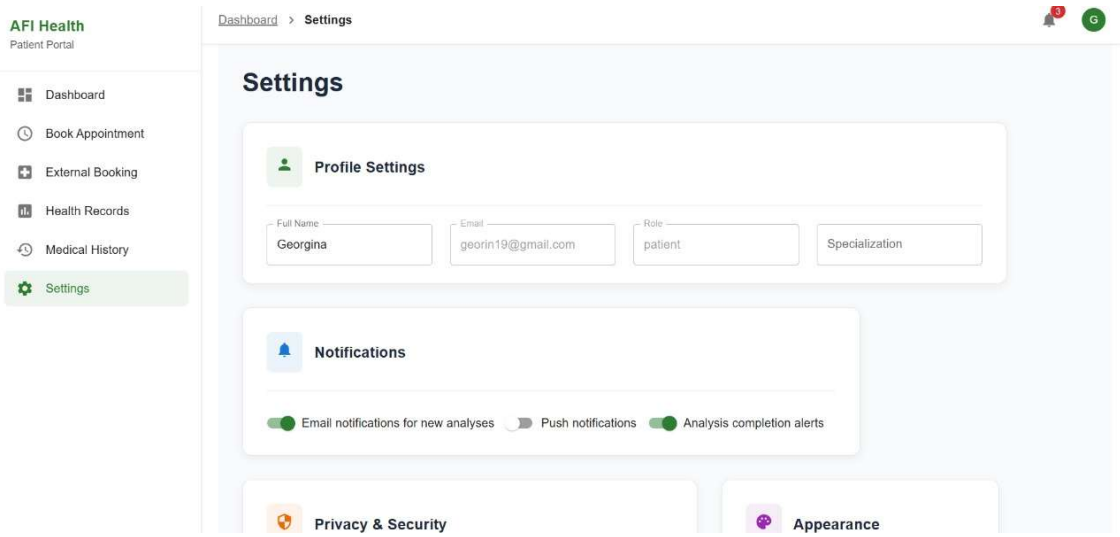


Fig 6.2.22 Settings Page

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

This AI-powered web platform revolutionizes amniotic fluid monitoring in maternal healthcare. By automating ultrasound analysis through deep learning, it delivers instant AFI classifications—identifying Low, Normal, and High levels with enhanced speed and accuracy.

The system integrates a smart TensorFlow model with a responsive Fast API backend and an intuitive React interface, creating a seamless experience for both doctors and patients. It eliminates traditional manual measurements, making diagnostics more reliable where specialist access is limited.

Beyond core analysis, the platform offers secure logins, prediction history, and appointment booking—forming a complete digital health solution. Built for scalability, it establishes a strong foundation for future AI-driven prenatal tools, bridging technology and clinical care to improve outcomes for mothers everywhere.

7.2 FUTURE SCOPE

To overcome existing limitations and extend system functionality, the following enhancements are proposed:

1 Add Quadrant-Based AFI Measurement

- Integrate advanced segmentation models (U-Net, Mask R-CNN) to:
  - Segment amniotic fluid pockets
  - Calculate AFI numerically
  - Provide both classification + actual AFI value.

2 Incorporate Doppler Ultrasound Features

- Adding Doppler metrics can enhance diagnosis of fetal distress or placental insufficiency.

3 Add CTG (Cardiotocography) Analysis

- Combine AFI + fetal heart rate patterns → comprehensive fetal health assessment.

4 Doctor–Patient Communication System

- A messaging or chat module to clarify reports or book teleconsultations.

5 Model Optimization for Mobile Use

- Deploy lightweight TensorFlow Lite or ONNX models for:
  - Smartphone-based clinical screening.
  - Offline predictions.

6 Integration with Hospital Information Systems (HIS)

- To allow seamless:
  - Record transfer
  - Patient management
  - Report integration

7 Multi-Language Support

- Important for usage in rural areas.
- Kannada + English + Hindi interfaces can be provided.

8 Add Explainable AI (XAI)

- Use Grad-CAM to show:
  - Which regions of the ultrasound image influenced the prediction.
  - Improve transparency for doctors.

9 Real-Time Monitoring System

- A live tracking dashboard for doctors to monitor multiple patients simultaneously.

BIBLIOGRAPHY

1. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson Education.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
3. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.
4. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
5. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
6. Tanenbaum, A. S. (2016). *Modern Operating Systems*. Pearson.
7. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
8. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill.
9. Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.
10. Silberschatz, A., Korth, H., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
11. Phelan, J. P., & Smith, C. V. (1987). Amniotic Fluid Volume: Physiology and Clinical Significance. *American Journal of Obstetrics and Genecology*.
12. Dashe, J. S., Pressman, E. K., et al. (2000). Relationship Between Ultrasound-Measured Amniotic Fluid Volume and Fetal Outcome. *Obstetrics & gynaecology*.
13. Nabhan, A. F., & Abdelmoula, Y. A. (2009). Amniotic Fluid Index vs. Single Deepest Pocket Measurement. *International Journal of Gynecology & Obstetrics*.
14. Yu, L., Wang, S., & Thompson, P. (2018). Convolutional Neural Networks for Medical Image Classification. *Nature Biomedical Engineering*.
15. Litjens, G., et al. (2017). A Survey on Deep Learning in Medical Image Analysis. *Medical Image Analysis*.